

Planet Blitz — AI 활용 기술 문서

NAN 2026 Game×AI 해커톤 사전 과제 · 개인 참가 김민석 (v0o0v2@gmail.com) · 2026-08-10

0. 요약 — 무엇을 만들었고, 무엇을 지휘했는가

Planet Blitz는 1인이 26일 동안 만든 온라인 게임이고, 게임 코드와 테스트 중 사람이 직접 타이핑한 줄은 없습니다. 전부 Claude Code가 작성했습니다. 제가 맡은 일은 AI를 지휘하는 체계를 설계하고 운영하는 쪽이었습니다. 요구를 확정하는 인터뷰, 결정을 붙잡아 두는 ADR, 작성자와 비평자를 갈라 놓은 멀티에이전트 파이프라인, AI 산출물을 기계적으로 걸러 내는 검증 게이트가 그 체계를 이룹니다.

그 체계가 낸 결과의 규모(2026-07-15 ~ 08-09 실측, public 리포에서 직접 확인 가능):

지표	값
커밋 / 머지된 PR	1,392 커밋 / 498 PR
TypeScript	872파일 · 약 29만 줄
자동 테스트	378파일 · 8,676건 전량 통과 (약 82초)
아키텍처 결정 기록(ADR)	55건
DB 마이그레이션 / Edge Function	63건 / 3종 (서버 권위 쓰기)
픽셀아트 에셋	PNG 573장 (AI 생성)

전량 AI 작성이라는 말은 직접 세어서 확인하실 수 있습니다. 총 1,392 커밋은 비머지 894건과 머지 498건으로 나뉘는데, 비머지 894건 가운데 849건(95%)이 Co-Authored-By: Claude trailer를 담니다. trailer가 없는 45건도 같은 AI 워크플로의 산출이고, 일부 세션 설정에서 trailer가 누락됐을 뿐입니다. 아래 명령으로 세어 보실 수 있습니다(2026-08-09 기준).

```
git rev-list --count --no-merges HEAD # 894
git log --no-merges --grep="Co-Authored-By: Claude" --format=%H | wc -l # 849
```

1. 구조 설명 — AI가 감당할 수 있게 설계한 아키텍처

AI에게 대량의 코드를 맡길 때 가장 큰 위험은 그럴듯하지만 틀린 코드입니다. 이 프로젝트는 아키텍처 자체를 기계가 검증할 수 있는 형태로 설계해 그 위험을 구조적으로 낮췄습니다.

- 결정론 시뮬 코어**(src/sim)는 순수 TypeScript로, 시드 RNG만 씁니다. 렌더·시간·난수 API를 쓰면 ESLint 에러가 납니다. 지켜 달라고 부탁하는 대신 빌드가 막게 했습니다.
- 렌더 계층**(PixiJS v8)은 시뮬이 뱉은 스냅샷만 소비하고 거꾸로 흘러들지 못합니다. 화면 버그가 게임 로직을 오염시킬 수 없습니다.
- 서버**(Supabase Postgres + Edge Functions)는 보상 발급과 래더 변동 같은 이득 쓰기를 service_role로만 처리합니다. AI가 짠 클라이언트를 신뢰하지 않는다는 전제 위에 온라인 게임을 올렸습니다.

결정론은 그대로 테스트 가능성이 됩니다. 같은 시드에서 같은 결과가 보장되므로, AI가 고친 코드가 게임 결과를 바꿨는지를 틱 단위 상태 해시 비교로 기계가 판정합니다. 사람이 눈으로 확인하지 않아도 되니 AI의

대량 산출을 받아 낼 수 있습니다.

서버가 리플레이를 전수 재실행해 검증하던 설계는 의도적으로 건너뛴습니다(ADR-0050). 재실행은 서버가 시물 전체를 실행시켜 배포 결함을 키우는 데 비해 막아 주는 폭이 좁았습니다. 대신 보상은 서버가 자기 시드로 직접 굴리고, 획득 원장과 개연성 캡으로 위조 이득을 정직한 최상위 플레이어 수준 안에 가두는 쪽으로 바꿨습니다. 시물 결정론 자체는 기본 스위트의 상시 게이트로 남아 있습니다

(`tests/determinismGate.test.ts` 가 같은 시드를 두 번 돌려 상태 해시가 원소 단위로 같은지 검사합니다).

2. AI 디렉팅 파이프라인 — 사람은 무엇을 했는가

2-1. 설계: 인터뷰로 확정하고, ADR로 고정한다

- 개발 첫날 AI와 딥 인터뷰를 12~14라운드 돌려 게임 설계 문서(GDD)를 확정했습니다. AI가 모호한 지점을 소크라테스식으로 추궁하면 사람이 결정하고, 그 결과가 `docs/GDD.md` 로 남습니다.
- 되돌리기 어려운 결정은 ADR 55건으로 고정했습니다. AI 세션은 매번 백지에서 시작하므로, 왜 이렇게 했는지를 리포에 적어 두는 일이 AI의 장기 기억 역할을 합니다.

2-2. 실행: 병렬 레인 + 작성자/비평가 분리

- 작업은 `git worktree`로 격리한 병렬 레인에서 진행했습니다. 여러 AI 세션이 동시에 서로 다른 기능(스킬 210종, 촉매 배선, 침공 HUD 등)을 각자 브랜치와 PR로 밀고 나갑니다.
- 한 레인 안에서도 역할을 나눈 서브에이전트를 씁니다. 탐색, 계획, 구현, 비평, 검증 순서이고, 특히 작성자가 자기 결과를 스스로 승인하지 못하도록 별도 컨텍스트의 비평자가 결함을 적발합니다. 비평이 반복해서 잡아낸 결함 패턴(항진 명제 테스트, 하드코딩 답습 등)은 문서로 쌓아 다음 레인의 지시문에 되먹였습니다.

2-3. 사례 — 산출물 대신 채점 기준을 고쳤다

디렉팅이 실제로 어떤 일이었는지 한 건으로 보이겠습니다.

전체 테스트 스위트에서 12건이 상시 빨간 채로 방치돼 있었습니다. 원인을 파 보니 단언이 붓이 런을 완주하는지에 걸려 있어서, 밸런스 패치로 피격 피해 배수가 바뀐 순간 드랍과 경험치 배선을 재려던 테스트들이 썰 것을 못 재고 있었습니다. 개별 테스트를 손보는 대신 원칙을 세웠습니다. 단언이 "붓이 이길 수 있는가"에 의존하면 게이트에서 내린다는 것입니다(ADR-0051). 그 축을 내리되 배선 증명은 내구 파일럿과 최소 틱으로 다시 써서 살렸습니다. 지우고 도망친 것은 아닙니다. 그 결과 스위트가 241초에서 55초로 줄고 main이 전량 초록이 됐습니다.

ADR도 채점 대상이었습니다. ADR-0050은 구현에 착수하기 직전 전수조사에서 자신의 전제 셋이 코드와 어긋난다는 사실이 드러나, 정정 절을 달고 집행됐습니다. 그대로 밀어붙였다면 아무것도 막지 못하는 문을 잠글 뻔했고, 결정은 유지하되 표적만 다시 잡았습니다. 이런 자기 정정 기록을 지우지 않고 남겨 두는 것이 AI 세션들의 같은 실수를 막아 주었습니다.

2-4. 검증: AI 산출물은 게이트를 통과해야 다음으로 간다

- 검증 규약을 지시문으로 못박았습니다. 편집 중에는 짝 테스트, 커밋 전에는 변경분 테스트, PR 전에는 전체 스위트와 타입 검사와 린트를 돌립니다. 리포 `CLAUDE.md` 에 적어 두어 모든 AI 세션에 자동으로 주입됩니다.

- **결정론 게이트**는 대표 행성과 빌드 조합을 같은 시드로 두 번 돌려 해시가 맞는지 상시 검사합니다. 골든 상수와 대조하는 대신 자기 자신과 대조하므로 유지비 없이 재현성을 지킵니다.
- **밸런스 계측 CLI**(`bench/`)는 클리어율과 구간 틱, 드랍량을 결정론 봇으로 일괄 측정합니다. AI가 수치를 바꾸면 사람은 감이 아니라 표를 보고 판단합니다.
- 세션이 끝날 때마다 무엇이 틀렸고 왜 틀렸는지를 메모리 문서로 남겨 다음 세션에 주입했습니다. "파이프가 `exit code`를 덮어 거짓 초록을 만든다" 같은 기록인데, 같은 실수를 두 번 하지 않게 하는 이 되먹임이 26일 완주의 실질적인 동력이었습니다.

3. 주요 프롬프트·지시 사례 (실물 — 게임 리포에서 확인 가능)

① **검증 규약을 지시문으로.** 리포 `CLAUDE.md` 에 있고 모든 AI 세션에 자동 주입됩니다.

"전체 스위트를 편집 한 건마다 돌리지 않는다. 실측으로 비용 차이가 34배다. (...) `pnpm verify` 를 파이프에 물리지 마라(`| tail` 금지) — `exit code`가 `tail` 것이 되어 거짓 그린이 된다."

이 지시가 들어간 뒤로 AI 세션의 검증 시간이 편집당 수 분에서 수 초로 내려갔고, 파이프 때문에 생기던 거짓 성공 보고가 사라졌습니다.

② **검증의 기준을 원칙 문장으로.** 같은 문서에 있습니다.

"골든 재생성은 검증이 아니다. 다시 뜯 값으로 그 값을 검사하면 항진이다. (...) 이 파일은 골든 상수를 갖지 않는다 — 모든 단언이 '같은 입력을 두 번 돌린 결과끼리' 비교다."

AI가 테스트를 고치는 가장 쉬운 길은 기대값을 실측값으로 덮어쓰는 것입니다. 이 문장이 그 길을 원칙 수준에서 막았고, 덕분에 수천 건 테스트의 신뢰가 유지됐습니다.

에셋 생성 프롬프트도 전부 기록해 두었습니다. 베르단 돌격몹을 만든 `insectoid darting larva charger` (top-down 64px)처럼, 에셋 라이브러리 인덱스에 974건이 프롬프트와 라이선스, 해시와 함께 남아 있습니다(\$5).

4. 사용한 AI 도구

도구	용도
Claude Code (Anthropic)	게임 코드·테스트·문서 전량 작성, 멀티에이전트 오케스트레이션
oh-my-claudecode	에이전트 역할 분리(탐색·계획·구현·비평·검증)와 병렬 레인 운영
PixelLab	픽셀아트 에셋 573장 생성 (기체·적·보스·아이콘·타일·UI)
Meshy	보스 3D 모델 생성 (타이틀·연출용)

비-AI 스택: TypeScript · PixiJS v8 · three.js · Vite · Vitest · Supabase(Postgres + Edge Functions) · GitHub Actions(CI·Pages 배포).

5. 외부 에셋 · 오픈소스 출처

자산	출처 · 라이선스
BGM 6트랙	Juhani Junkala · congusbongus · Emma_MA (OpenGameArt) — 전부 CC0 1.0. 트랙별 출처 링크: 리포 <code>assets/audio/CREDITS.md</code>
효과음 9종	Kenney.nl 「Sci-Fi Sounds」·「Interface Sounds」 — CC0 1.0, 동일 문서에 명시
픽셀아트	Pixellab 생성물 — 상업 이용 가능 (약관: https://pixellab.ai/termsofservice)
3D 모델	Meshy 생성물 — 상업 이용 가능 (meshy.ai 이용약관)

음악만은 AI 생성물을 의도적으로 배제했습니다. 학습 데이터의 저작권이 불확실하다고 판단해서인데, 이 정책은 `assets/audio/CREDITS.md` 에 적어 두었고 전 트랙이 사람이 작곡한 CC0 음원입니다.

오픈소스 의존성은 전부 MIT입니다. `pixijs`, `three`, `@supabase/supabase-js`, `vite`, `vitest`, `eslint` 등이고 전체 목록은 `package.json` 에 있습니다. 게임 소스 자체는 독점 라이선스지만 리포가 공개돼 있어 열람은 자유롭고, 플레이는 배포 링크로 하시면 됩니다(「게임 소개 문서」 §2).

6. 다음 단계 — 이 방식이 아직 못 하는 것

26일 동안 확인한 것은 하나입니다. 기계가 판정할 수 있게 아키텍처를 설계하면, AI 산출량의 상한이 사람의 검토 속도에서 게이트의 처리량으로 옮겨 갑니다. 남은 병목은 셋인데, 셋 다 AI의 다음 단계가 풀어야 할 문제라고 봅니다.

- 재미는 아직 기계가 못 잹니다.** 결정론 봇으로 클리어율과 구간 틱은 재지만 재미있는지는 재지 못해서, 밸런스는 여전히 사람이 표를 보고 판단합니다. 다음 단계는 플레이 텔레메트리를 게이트로 승격시키는 일입니다.
- 비평자가 사람의 취향을 대리하지 못합니다.** 시각 품질을 판정할 때 비평 에이전트가 세운 지표 다섯을 사람이 통째로 뒤집은 적이 있습니다. 심미 판단을 위임하는 문제는 아직 열려 있습니다.
- 세션 간 기억이 아직 수동입니다.** 실패 패턴을 사람이 손수 메모리 문서로 되먹이고 있는데, 이 루프를 자동화하는 것이 다음 레버입니다.

셋 다 사람이 아직 대신해 주고 있는 판정입니다. 제가 다음에 설계하고 싶은 것도 게임보다는, 이 판정들을 기계에 넘기는 다음 게이트입니다.